AIPL / ABCL^c+ User's Guide

— syntax, usage, and sample programs —

ABCL^c+ / AIOS implementation (`yaskodama/abclcp`, branch `make-src-base`)

2026-08-31 edition

Target: branch `make-src-base` (the current specification, including the seven disciplines)

Abstract

This is the **current** user's guide to AIPL (ABCL^c+). Part I gives the shape of the language and the shortest path to running something; Part II specifies the syntax and the type system; Part III covers the seven disciplines that sit on top of it; Part V walks through the sample programs with their sources and their actual output; Part VI describes the tools for building, running and checking; Part VII collects the grammar summary, the present limits, and the migration notes from the previous edition.

AIPL is a concurrent language whose unit is the **actor**. A value is not returned with `return` but with `reply`. Three static mechanisms sit on top of that — **types** (the return type is inferred from `reply`, and can also be annotated), **effects** (a method declares what it does to the world, as in `!{ai, net}`), and **deadlines** (`now ... timeout ... else ...` bounds every wait). For example, "a real-time actor synchronously calls a model outside the box" can be forbidden by the effect check alone.

Above those sit the **seven disciplines** (Part III) — the failure type `result`$\langle\tau\rangle$, first-class reply destinations, resource ordering, obligation levels, and session types. All of them are built so that they **work without you writing annotations**. As a result, **as long as the target of a wait is known as a class, or the level of the target node is declared, deadlock freedom is guaranteed by the type system**. That is not a claim on paper only: it is machine-checked in Coq/Rocq.

What this guide describes is the **core** — the part that all three implementations (OCaml, Python, JavaScript) provide. The Python implementation additionally has **extensions** (record types, refinement types, channels, ownership), so the language has two layers (§44).

If you only want the delta from the previous edition (2026-07-30), read §44. `become` has been **removed from the language**.

Contents

Part I
Getting started

1 What AIPL is

AIPL is an actor language in the lineage of ABCL/1. A program is a sequence of **class declarations** and **global statements**; actors created from the classes run by sending messages to each other.

```
class Counter {
  var n = 0;                        // a field (the actor's state)
  method bump() : int !{mut} {      // return type int, effect mut
    n = n + 1;
    reply(n);                       // hand a value back to the caller
  }
}

var c = new Counter();              // create one actor
print(now c.bump() timeout 100 else 0);
```

Four differences from other languages are worth learning first.

1. **One actor is one thread**, and it processes the messages it receives one at a time. Fields are never touched concurrently.

2. **Values come back through `reply`.** There is no `return`. `reply` resolves the caller's future; the method keeps running afterwards.

3. **There are three kinds of send.** `send` (does not wait; a statement), `now` (waits for the reply; an expression), and `future` (does not wait; hands you a ticket; an expression).

4. **Waiting freezes you.** While an actor waits in a `now` it cannot process any other message. That is why waits carry deadlines.

2 Three static mechanisms

2.1 Types — what you give back

A method's return type is inferred from the `reply` calls in its body. The optional annotation `method m(x) : T` may also be written. When you write it, the checker verifies that every execution path replies, and the declared type flows into expression inference as the *expected* type.

2.2 Effects — what you do to the world

A method declares its effects with `{...}`. There are eight of them (§11). If you omit the annotation the effects are inferred; if you write it, "the effects of the body $\subseteq$ the effects you declared" is checked.

```
method plan(goal) : string !{ai, net} { reply(ai_call("plan: " ++ goal)); }
method peek()     : int    !{}        { reply(n); }
```

The point is that `ai` and `net` are separate: on-device inference that "uses AI but never leaves the box" is expressed with `{ai}` alone.

2.3 Deadlines — how long you will wait

```
var r = now planner.plan(g) timeout 2000 else "(timed out)";
```

If no reply arrives in time, the value of the `else` branch is used. If you write no `else`, the expression has type `result`$\langle\tau\rangle$ and cannot be used before it is checked (§12).

2.4 And seven disciplines on top

Types, effects and deadlines are the ground floor. Above them sit the failure type, first-class reply destinations, resource ordering, obligation levels and session types (Part III). All of them are built so that they **work without annotations**. As a result, **as long as the target of a wait is known as a class, or the level of the target node is declared, deadlock freedom is guaranteed by the type system** (§15).

3 Running something in five minutes

3.1 Building the implementation

```
cd ~/aios/abclcp
make -C src thread_repl
```

Building with `dune` fails because `src/lexer.ml` is generated twice, so use `src/Makefile`.

3.2 Running a program

Pass a file of REPL commands with `-f`.

```
cat > run.repl <<'EOF'
load docs/samples/guide/g2_now_future.aipl
compile
EOF
./src/abclrepl_thread -q -f run.repl < /dev/null
```

```
twice = 43
awaited = 20
v=7
```

Do not pass a `.aipl` file to `-f` directly. It would be read line by line as REPL commands, the class definitions would never be loaded, and you would get `Actor xxx not found`. Always write `load` and `compile`.

3.3 Running only the type checker

The REPL depends on SDL, so a small separate driver exists for type checking alone (§36). Once built, `./tc file.aipl` prints the table of return types, effects and signatures.

```
$ ./tc docs/samples/guide/g6_effects.aipl
=== g6_effects.aipl ===
[OK] type check passed
[method return types (inferred from reply)]
  Counter#bump -> int
  Planner#plan -> string
  ViaNow#run -> string
[method effects (inferred / declared)]
  Counter#bump !{mut}
  Planner#plan !{ai, net}
  ViaNow#run !{ai, net}
```

Part II
Language guide

4 The shape of a program

A program is a **sequence of declarations**. A declaration is one of the following; the order is free, and names are resolved at run time.

Declaration	Meaning
`class C { ... }`	class definition
`var x = e;`	global variable
`var x = new C(args);`	create an actor
`send t.m(args);`	global asynchronous send
`send! t.m(args);`	the same, with the check relaxed
`f(args);`	call a builtin function

> `if` and `while` cannot appear at global level. If you need control flow, put it inside a method and start it with `send`.

A class is a **sequence of fields** followed by a **sequence of methods**.

```
class Counter {
  var n = 0;                    // fields come first; declare with var or float
  float ratio = 1.5;
  method init(start) {          // init is called automatically by new
    n = start;
  }
  method bump() : int !{mut} {
    n = n + 1;
    reply(n);
  }
}
```

- Fields come **before** methods.

- A class needs **at least one method** (the grammar has no field-only class).
- A method named `init` is called automatically by `new C(args)`. If the number of arguments does not match, creation is skipped and `[Actor] x: no init; skipped` is printed.
- A field's type is fixed by its initializer. `var n = 0;` is `int`, so assigning a `float` or a string to it later is a type error.

5 Lexical structure

Kind	Content
Comments	`// to end of line`, `/* ... */` (not nestable)
Keywords	`class`, `method`, `var`, `float`, `new`, `send`, `send!`, `now`, `future`, `await`, `if`, `else`, `while`, `do`, `select`, `case`, `timeout`, `call`, `replyto`, `self`, `sender`, `remote`, `true`, `false`
Integers	`0`, `123`
Floats	`1.5`, `100.` (a trailing dot alone makes it `float`)
Strings	`"..."`, with `\\` `\"` `\n` `\t` `\r`
Booleans	`true`, `false`
Identifiers	`[A-Za-z_][A-Za-z0-9_]*`

`float` is a keyword, so it cannot be used freely as a variable or type name (it *can* be written as a return annotation, `: float`, which has its own grammar rule).

6 Types

Type	Description
`int`, `float`, `string`, `bool`, `unit`	base types
`T[]`	array (handled through the `array_*` builtins)
`future T`	the result of a `future` send; `await` takes the `T` out
`actor(C)`	an actor of class `C`; the type of `new C(...)`
`any`	a boundary whose content is not statically guaranteed (`sender`, remote targe

A return annotation may be `int` / `float` / `string` / `bool` / `unit` / `any`, or a class name starting with a capital letter (an actor type). An unknown lowercase name is a syntax error: `unknown type name in return annotation`.

Method parameters are monomorphic. The type variables of a signature are shared by every call site, so calling the same method with `int` and with `string` conflicts. Polymorphic methods cannot be written.

There is **no implicit conversion** between `int` and `float`. If `var h = new Hello(5);` feeds a `float` field you get a type error; write `new Hello(5.)`.

7 Expressions

Expression	Meaning
`123`, `1.5`, `"abc"`, `true`, `false`	literals
`x`	variable, field or parameter
`- e`	unary minus; lowers to `neg(e)`. `int/float`
`a ++ b`	string concatenation (both sides are stringified)
`a + b`, `a - b`, `a * b`	arithmetic. `int` with `int` gives `int`; `float` with `float` gives `float`
`a / b`	division. Even `int` with `int` gives `float` (no integer division)
`a == b`, `a != b`	equality. `int/float/string/bool`
`a > b`, `a < b`, `a >= b`, `a <= b`	comparison; the result is `bool`
`new C(args)`	create an actor (effect `{mem}`)
`now t.m(args)`	synchronous send: wait for the reply and become that value
`now t.m(args) timeout` n `else` e	synchronous send with a deadline
`future t.m(args)`	asynchronous send; returns `future T`
`await e`	wait for a future to resolve
`await e timeout` n `else` e'	`await` with a deadline
`f(args)`	call a builtin function
`( e )`	parentheses

The target `t` is **a variable holding an actor reference**, or `remote("host:port", "actorName")`. It is not a class name.

7.1 Precedence

From weakest to strongest:

equality `== !=` < comparison `> < >= <=` < concatenation `++` < additive `+ -` < multiplicative `* /` < unary minus

So `1 + 2 == 3` reads as `(1 + 2) == 3`, and `"x=" ++ a + b` reads as `"x=" ++ (a + b)`. Every binary operator is **left associative** (equality included) — `1 == 2 == false` reads as `((1 == 2) == false)` and evaluates to `true`. Unary minus is a prefix, so it can be stacked (`- - 3` is 3).

7.2 Do not confuse + with ++

`+` is **numeric only**. Mixing in a string gives `no overload of + matches (string, int)`.

```
print("count = " ++ n);      // correct
print("count = " + n);       // type error
```

`++` stringifies both sides first, so neither side has to be a string.

7.3 Ambiguous overloads

When both operands are unbound, `a + b` could be `int` or `float`: it has no principal type. That is an error. An annotation makes the expected type flow into the expression and settles it.

```
method add(a, b)       { reply(a + b); }   // error: ambiguous overload
method add(a, b) : int { reply(a + b); }   // the annotation settles it
```

`AIOS_LAX_OVERLOAD=1` restores the old behaviour (warn and pick the default candidate). Equality `==` and `!=` have four candidates but all of them return `bool`, so they are never ambiguous (ambiguity is only reported when the *return types* disagree).

8 Statements

Statement	Meaning
`var x = e;`	variable declaration
`var x = new C(args);`	declaration that creates an actor
`x = e;`	assignment
`f(args);  call f(args);`	call a builtin function
`send t.m(args);`	asynchronous send; does not wait for a reply
`send self.m(args);`	asynchronous send to yourself
`send sender.m(args);`	asynchronous send to the sender
`send! t.m(args);`	send with the check relaxed
`if (e) s  if (e) s else s`	conditional (the parentheses are required)
`while e do s`	loop (no parentheses required)
`{ s ... }`	block
`select { ... }`	selective receive

`now self.m()` and `now sender.m()` **cannot be written** — the grammar forbids them. A `now` to yourself is a self-deadlock (you are busy running that very method, so nobody is left to produce the reply), and not being able to write it is exactly the safe side. If you want to factor out a computation such as inverse kinematics, put it in another actor and call `now kin.solve(...)`.

A variable declaration with a type annotation, `var x : T = e;`, does **not exist yet**. It is in the design notes but unimplemented.

9 Actors and communication

9.1 The three sends

	Waits?	Statement or expression	Inherits the callee's effects?
`send a.m(x);`	no	statement	no
`now a.m(x)`	until reply	expression (of the declared/inferred type)	**yes**
`future a.m(x)`	no	expression (`future T`)	no

"Only the side that waits carries the callee's effects" is the rule of effect propagation (§11).

9.2 `reply`

A method does not "return" a value; `reply(v)` resolves the caller's future. The type system gives each method a return type, and every `reply` in the body together with every `now` / `future` call site shares that type.

- `reply` happens **at most once**. A second one would arrive after the waiter already took the first value, so it would be silently dropped. That is made a type error.
- If the return type is *declared* as something other than `unit`, the checker verifies that **every execution path replies exactly once**. The body of a `while` may run zero times, so it is conservatively judged as "there is a path that does not reply".
- A method with no `reply` at all is treated as returning `unit`. Notification-style methods called with `send` are of this kind.

```
method m(k) : int { reply(k); reply(k + 1); }                    // type error
method m(k) : int { if (k > 0) { reply(1); } else { reply(2); } } // OK
```

9.3 `select`

```
select {
  case job(k) -> { reply("done:" ++ k); }
  timeout 500 -> { print("timed out"); }
}
```

`case m(x1, ..., xn)` is matched against the signature of the method of the same name. A mismatch in the number or type of arguments is a type error. `timeout` is in milliseconds and may be omitted.

A `reply` in a case body answers "the selected message", not the enclosing method. The implementation switches the reply destination to the selected message before running the case, so the `reply` above answers `job`. Hence a method containing a `select` may itself be `unit`. The `timeout` body, by contrast, runs with the enclosing method's reply destination.

> `select` looks for messages that have *not yet been processed*. If ordinary method dispatch takes the message of that name first, nothing is left for the case, so it is safer to design the names you `select` on so that nobody calls them directly (see the output in §26).

10 Waiting with a deadline

10.1 Why it is needed

A `now` or `await` without a deadline waits forever if no reply comes. An actor is one thread and a wait blocks on a condition variable, so while it waits it cannot process any other message. It turns to stone.

```
now t.m(a) timeout <milliseconds> else <expression>
await f    timeout <milliseconds> else <expression>
```

If the reply arrives in time you get its value; otherwise you get the value of the `else` branch.

10.2 The rules

- The `else` branch must have **the same type as the body**, because a timeout must not change the type of the whole expression. Otherwise: `now: the else branch has type string but the call returns int`.
- The deadline must be **positive** (`timeout must be positive`).
- A `now` / `await` without a deadline is a **warning** by default, and an **error** under `AIOS_STRICT_DEADLINE=1`.
- The upper bound on the number of replies is the `max` of the body and the `else` branch (not their sum), since only *one* of them runs.
- A rejected future (`reject`) is distinguished from a timeout and raises an exception.

In the implementation, OCaml's `Condition` has no timed wait, so the deadline is measured by polling at 1 ms granularity.

10.3 Relation to type soundness

Writing a deadline makes every wait finish in finite time. But that only means *a deadlock is converted into a failure in finite time*; the failure does not disappear. Two actors waiting on each other with `now` both time out. Nothing is stuck — and nothing is right either.

To say **that nothing gets stuck** in the type system you need not deadlines but **obligation levels** (§15). Those are **machine-checked in Coq/Rocq**: for the core *including* `await`, type

soundness, type safety and deadlock freedom are proved with no axioms (`coq/AIPLSoundness2.v`). The first version could state deadlock freedom only for a fragment with `await` removed, which is what the second version fixes.

The progress theorem changed as follows.

	What progress states
1st version	terminal state ∨ can step ∨ **all tasks are waiting**
2nd version	terminal state ∨ can step

Termination is still not claimed. `while 1 > 0 do {}` does not stop, but it is not a wait, so it does not contradict progress.

11 Effects

11.1 The eight effects

Effect	Meaning	Effect	Meaning
`mut`	assignment to a field	`net`	communication outside the node
`time`	reading the clock, waiting	`ai`	model inference
`io`	input/output to a device	`fs`	persistence
`mem`	dynamic allocation	`log`	output for observation only

This is not a newly designed taxonomy: it is the capability classification the evaluator already carried as run-time metadata, made static. The point is that `ai` and `net` are separate — a model call that leaves the box has both, while on-device inference gets `{ai}` only. "Uses AI but never leaves the box" becomes visible in the signature.

11.2 How to write them

```
method plan(goal) : string !{ai, net} { ... }   // after the return type
method peek()             !{}         { ... }   // explicitly effect-free
method tick()                         { ... }   // omitted: inferred
```

The annotation is optional (gradual). If you omit it, inference simply runs and nothing is said. If you write it, "the effects of the body ⊆ the effects declared" is checked. Declaring more than you use is allowed. An unknown effect name is rejected as a typo.

```
[Type error] line 1, col 18: unknown effect(s) network in A.m;
  known effects are mut, time, io, mem, net, ai, fs, log
```

11.3 What counts as an effect

Construct	Effect
assignment to a field	`{mut}`
assignment to a local	**not an effect**
`new C(...)`	`{mem}`
calling a builtin	that function's effects (§22)
send to a `remote(...)` target	`{net}`
local send with `now`	**inherits the callee's effects**
`send` / `future`	does not inherit

That `mut` covers only assignment to fields is deliberate: it lets a pure actor (safe to replicate) and an actor with external effects (exactly one at a time) be told apart by the effect set alone.

Propagation through `now` iterates to a fixpoint. Effects only grow, so it terminates.

11.4 What this lets you forbid

```
class Controller {
  method step(s) : string !{io, time} {
    var r = now planner.plan(s) timeout 100 else "";   // plan is !{ai, net}
    reply(r);
  }
}
```

```
[Type error] method Controller.step declares {io, time}
  but its body has {ai, net, io, time} (missing {ai, net})
```

"A real-time actor synchronously calls an agent outside the box" is forbidden by the effect check alone. What used to be stated as a design decision now holds mechanically in the type system.

> **A known hole.** Splitting a `now` into `future` and `await` *escapes* this check: the implementation adds the propagation edge only at the `now` site and not at the `await`. A reproduction is in `docs/samples/soundness2_effect_gap.aipl`. The fix (carrying effects in the `future` type) is described in the second soundness report.

Part III
The seven disciplines

On top of types, effects and deadlines sit seven disciplines. All of them are built so that they **work without annotations** — because the information they need is usually already written in the program.

Discipline	How you write it	What it stops
Effects	`!{ai, net}`	the real-time side waiting on a model outside the box
Deadlines	`timeout` n `else` e	waits that never return
Failure type	`timeout` n (no `else`)	letting a timed-out value flow on as a normal one
First-class reply	`replyto` / `answer` / parameter type `reply`	dropped replies and double replies
Resource order	`acquire` / `release` / `resource_order`	forgetting to release, double acquire, acquiring in reverse order
Obligation levels	`@`n (inferred if omitted)	cycles of waiting
Session types	`protocol_define` / `_start` / `_end`	steps taken out of order

12 Putting failure in the type — `result`$\langle\tau\rangle$

If you write `else` on a deadline, the timed-out value and the normal value have **the same type**. The program cannot ask whether -1 came back because it timed out or because -1 was computed.

Write no `else` and you get `result`$\langle\tau\rangle$.

How you write it	Type
`now a.m(x) timeout 200 else -1`	`int`
`now a.m(x) timeout 200`	`result`$\langle$`int`$\rangle$

Use `is_ok` / `timed_out` / `value` to take the value out.

```
var a = now s.fast(42) timeout 200;      // result<int>
if (is_ok(a)) { print("fast ok " ++ value(a, -1)); }
else          { print("fast timed out"); }
```

A `result`$\langle\tau\rangle$ cannot be used as a number. Feeding it into arithmetic without checking stops the program at compile time.

```
no overload of + matches (result int, int)
```

13 Making the reply destination a first-class value

`replyto` yields, as a value, the reply destination of the message currently being processed. Its type is `reply`$\langle\rho\rangle$ (where ρ is the method's return type), and `answer(r, v)` sends the reply. `reply(v)` is sugar for that.

With the parameter annotation `reply` you can hand it to another actor. A front desk takes the request and a specialist answers **directly**.

```
class Backend {
  method handle(r: reply, x) : unit !{} {
    answer(r, x * 2);            // answer Frontend's caller directly
  }
}
class Frontend {
  var b = new Backend();
  method ask(x) : int !{} {
    send b.handle(replyto, x);   // pass the destination on; do not reply here
  }
}
```

A reply destination is **linear**: using it exactly once is an obligation.

Rule	Content
Creation	`var r = replyto;` creates the obligation
Consumption	`answer(r, v)`, or passing it as a send argument, discharges it
Exactly once	the same `r` must not be used twice
Nothing left	no obligation may remain when the method returns
Branches	the two branches of an `if` must join in the same state

The check is closed within a method. A reply destination **cannot be kept in a field** (a field's type is fixed by its initializer). A bounded buffer that "collects senders and answers them later" therefore cannot be written directly in the core as it stands.

14 Resource order

Effects are a **set**, so they cannot express order; "take it and give it back" cannot be said. So the `acquire` / `release` pairs are tracked through the body instead.

```
class Bus {
  var n = 0;
  method read(addr) : int !{mut} {
    acquire("i2c");
    n = n + 1;
    var v = addr * 2;
```

```
    release("i2c");           // delete this line and it is a type error
    reply(v);
  }
}
```

There are three rules. Nothing may still be held when the method returns. You may not release what you do not hold. You may not acquire twice. The two branches of an `if` must join holding the same things, and the body of a `while` must not change what is held.

14.1 A global order — stopping reverse acquisition

Checking pairs is not enough. If two actors take the same two resources in **opposite order**, both pairings are correct, yet at run time they wait for each other.

No new annotation is needed. The nesting "s was acquired while r was held" *is* the claim $r < s$. The claims are collected from the whole program and checked for cycles.

```
resource order: cache -> db -> cache is circular;
  cache before db (in Reader.run); db before cache (in Writer.run).
  Acquiring in opposite orders deadlocks
```

Every edge of the cycle carries **where it came from**, so the two places that acquire in opposite order are named directly.

You may also write `resource_order("bus -> dev")` instead of relying on inference. A declaration simply enters the same table as an edge, so if the declaration and the actual acquisitions disagree, that shows up as a cycle too. The inferred order can be printed with `AIOS_SHOW_LEVELS=1`.

```
[resource] bus                  @0
[resource] dev                  @1
```

> Only `acquire` calls whose resource name is a **string literal** can be tracked. A name held in a variable is invisible statically, so the declared order is also enforced at run time (`AIOS_STRICT_RESOURCE=1` reports the places that could not be tracked).

15 Obligation levels — stopping cycles of waiting

There are four shapes in which a wait never returns: a cycle of waits, a wait that is never replied to, a `select` waiting for a message that never comes, and resources acquired in reverse order. **Obligation levels** address the first.

Each method gets a number, and **a wait must always go to a strictly larger number**.

```
class Leaf { method f(x)  : int @2 !{} { reply(x + 1); } }
class Mid  { var l = new Leaf();
             method g(x)  : int @1 !{} { reply(now l.f(x) timeout 200 else 0); } }
```

You do not have to write them. If you omit them, they are inferred by a fixpoint that pushes levels up along the wait edges. An explicit annotation is treated as a fixed point; if it cannot be satisfied, that is where the contradiction is.

```
obligation level: B9#g (@2) waits on A9#f (@2);
                  a wait must go to a strictly higher level
obligation levels do not converge; the wait graph has a cycle
```

The inferred levels can be printed with `AIOS_SHOW_LEVELS=1`.

15.1 Waits that cross nodes

In a mesh the target lives on another node and is invisible statically. So each node declares a **floor** for its levels, and a wait towards that node must go up.

```
node_level("rt0", 10);            // the real-time node is a leaf (high level)
```

```
obligation level: Ctl2#go (@7) waits on node rt0 (floor @1);
                  a wait across nodes must go up
```

This is exactly the same shape as checking effects at the border with `node_allow`.

15.2 How much is guaranteed

As long as the target of a wait is known as a class, or the level of the target node is declared, deadlock freedom is guaranteed by the type system. This is not a claim on paper only — it is machine-checked in Coq/Rocq for the core *including* `await` (`coq/AIPLSoundness2.v`, no axioms).

The conditions matter. A dynamic target is invisible. Levels are per method, so two instances of the same class are treated alike. Termination and starvation are separate problems.

16 Session types — the order of an exchange

"Open it, then read, then close it at the end" is a promise that neither deadlines nor the existence of a sender can express. AIPL already had **run-time** session protocols, so the type checker reads **the same declaration**. No new declaration is needed.

```
protocol_define("Pipeline", "f.get -> r.scale -> st.put");
var sid = protocol_start("Pipeline");

var a = now f.get(4)    timeout 200 else 0;
var b = now r.scale(a)  timeout 200 else 0;
var c = now st.put(b)   timeout 200 else 0;
protocol_end(sid);
```

Get the order wrong and it is reported before the program runs.

```
protocol Pipeline: expected r.scale but the program sends st.put here
protocol Pipeline is incomplete at protocol_end; next expected st.put
```

(An example of real output.)

```
protocol P29: expected b.step2 but the program sends c.step3 here
```

16.1 When the session crosses actors

In a real program the steps do not fit in one straight sequence: you call one front-desk actor, and the remaining steps happen inside its body.

A synchronous send (`now` / `aios_now`) **runs to completion before** the caller continues. So the callee's sequence of sends can be spliced in at the call site (behaviour expansion), and a session that crosses actors can still be matched as a single sequence.

Asynchronous sends (`send` / `future`) are not spliced in — there is no telling when they run. If the callee contains steps, completion is left to the run-time check (`AIOS_STRICT_PROTOCOL=1` reports it).

> Steps refer to roles **by name**, so **pass actors as arguments**. If you keep them in fields, the run-time name (`c#f`) and the static name (`f`) disagree and the step cannot point at the role.

17 The `select` discipline

Two things are required of a `select`.

1. **Write a deadline.** A `select` without a `timeout` clause waits forever for a message that may never come.
2. **Check that a sender exists.** If nobody in this program sends the `m` of `case m(...)`, it is either a typo or something you forgot to write.

```
select waits for W2.ghost but nothing in this program sends it
select without a timeout waits forever; write `timeout <ms> -> { ... }`
```

Both are warnings by default. When sends are dynamic, or the program is exposed to the outside, no judgement is possible, so the check is simply not made — **the language does not see the whole world**.

18 Mesh deployment

An actor's source is shipped to another node and is parsed and type-checked **at the destination** before it is instantiated. The effect annotations travel with the code and are matched against the receiving node's policy.

```
node_allow("rt0", "mut, log, time");     // effects the real-time node accepts
deploy("rt0", "Servo", "servo1");        // ship Servo's source to rt0 and instantiate
var d = now remote("rt0", "servo1").step(3) timeout 200 else -1;
```

```
[deploy] Servo -> rt0/servo1 (compiled at destination)
```

Ship code carrying `ai` or `net` to a real-time node and it is stopped at the border. Effects turn from a discipline inside one node into **immigration control between nodes**.

19 Escape hatches (environment variables)

Every default is on the strict side. Environment variables exist that loosen things, but they are not meant for everyday use. The list is in §41.

Part IV
Reference

20 What the type checker looks at

Check	Content
Method existence	does the local target have a method of that name
Argument count and type	do the actual arguments match what the body requires
`reply` type	do all `reply` calls have the same (declared) type
`reply` count	at most once; with an annotation, exactly once on every path
select matching	do the `case` arity and argument types match the signature
Annotations when exposed	do the methods of a `web_exposed` actor carry annotations
Overloads	is the candidate unique (ambiguity is an error)
Effects	do the declared effects cover the body's; are the effect names known
Deadlines	the type of the `else` branch, positivity, presence
Failure type	is a `result`$\langle\tau\rangle$ used without being checked
Reply linearity	is `replyto` used exactly once
`reply` totality	does a method waited on by `now`/`await` reply on every path
Resources	the `acquire`/`release` pairs and cycles in the acquisition order
Obligation levels	does every wait go up; the same across node borders
Sessions	is the order of sends the order of `protocol_define`
select	is there a deadline; does anybody send the awaited message
Assignment to undeclared names	is a name assigned to without a `var` declaration
Deployment border	do the shipped code's effects fit the receiver's policy

Representative messages:

```
no method foo in actor(C)
type mismatch                                  <- argument or assignment type
no overload of + matches (string, int)         <- + is numeric only
ambiguous overload of + for ('a, 'a): candidate return types are float / int
reply type mismatch: this method is declared to reply with int,
  but replies with string here
method m is declared to reply with int, but some execution path does not reply
method m may reply more than once on some path
select: case m binds 1 variable(s) but method m takes 2
actor C is reachable from outside this program, but m has no return type
  annotation; ...
method C.m declares {log, time} but its body has {ai, log, net}
  (missing {ai, net})
unknown effect(s) network in A.m; known effects are mut, time, io, mem, ...
now: the else branch has type string but the call returns int
now: timeout must be positive
now without a deadline waits forever;
  write `now ... timeout <ms> else <expr>`      <- warning by default
assignment to undeclared name: cout (write `var cout = ...` to declare it)
no overload of + matches (result int, int)     <- check a result before using it
method Silent4.f is waited on by now/await but does not reply on every path
resource i2c is released without being acquired
method Dev4.use returns while still holding i2c
resource order: cache -> db -> cache is circular; ...
obligation level: B9#g (@2) waits on A9#f (@2);
  a wait must go to a strictly higher level
obligation levels do not converge; the wait graph has a cycle
protocol P29: expected b.step2 but the program sends c.step3 here
select waits for W2.ghost but nothing in this program sends it   <- warning
select without a timeout waits forever                            <- warning
```

```
deploy: Servo.step is @3 but node rt0 has floor @10
```

21 Exposing to the outside

`web_listen(port)` starts an HTTP gateway, and `web_expose(path, actorName)` gives an exposed endpoint an alias.

Endpoint	Content
`GET /`	a small UI
`POST /api/send`	`to=<actor>&method=<m>&args=<a,b>` reaches any actor
`POST /api/x/<key>`	reaches the alias registered with `web_expose`

`POST /api/send` **can reach any actor by name**. In other words it is `web_listen` that opens the border; `web_expose` only adds an alias. The type checker takes that into account: for an actor named by `web_expose`, a return annotation on every method (except `init`) is **mandatory**, and with `web_listen` alone it is a **warning**. The far side cannot see the body, so the reply type can only travel as a declaration.

`AIOS_LAX_EXPOSE=1` downgrades the error to a warning.

Arguments arriving over HTTP are built by interpreting the string as `int` → `float` → `string`, in that order. An annotation cannot make the boundary less dynamic, but it can declare *which type you promise*.

22 Builtin functions and their effects

Group	Names	Effect
Math	`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sqrt`, `exp`, `log10`, `abs`, `floor`, `ceil`, `round`	{}
Basic	`typeof`, `reply`, `neg`	{}
Output	`print`	{log}
Time	`wait`	{time}
Arrays (read)	`array_get`, `array_len`	{}
Arrays (allocate)	`array_empty`, `array_push`, `array_set`	{mem}
Actor creation	`spawn`	{mem}
Drawing (SDL)	`sdl_init`, `sdl_clear`, `sdl_line`, `sdl_erase_line`, `sdl_present`, `sdl_line_c`	{io}
Memory and tasks	`aios_memory_put/get/has/keys`, `aios_task_create/set/get/info`, `aios_tasks`	{fs}
Model inference	`ai_call`, `ai_call_with_system`, `model_generate`, `gemini_generate`, `openai_generate`	{ai, net}
Communication and exposure	`web_listen`, `web_expose`, `aios_now`, `aios_future`, `remote_review`, `remote_review_ja`, `remote_reviewer_host`	{net}
Introspection	`capabilities`, `capability_prims`, `aios_kernel`, `aios_actors`, `aios_actor_info`, `aios_actor_methods`, `aios_mailbox_len`, `actor_dump`	{log}
Events	`aios_emit`, `aios_events`, `aios_events_since`, `aios_event_count`	{log}
Protocols	`protocol_define/start/use/current/state/end/events`	{log}
Services	`aios_register_service`, `aios_services`, `aios_service_actor`, `aios_service_info`	{log}

A primitive that is not in this table is treated as effect-free.

Part V
Sample programs

The samples in this part are the ten programs in `docs/samples/guide/`. All of them type-check, their output is exactly as printed here, and **the three implementations (OCaml, Python, JavaScript) agree on that output** — verified with `tools/run_all_impls.sh`. (The sources in the repository carry Japanese comments; the listings below are the same programs with the comments translated.)

Samples for the more advanced features (`result`⟨τ⟩, delegating a reply destination, resource order, obligation levels, session types, mesh deployment) are the twenty programs in `docs/samples/paper/`; the examples in Part III come from there.

23 g1: classes, fields, `send`, string concatenation

```
// g1: class, field, init, method, send, string concatenation
class Greeter {
  var count = 0;              // a field, initialized as int

  method init(name) {         // called automatically by new
    print("hello, " ++ name); // ++ concatenates (both sides are stringified)
  }

  method tick() : unit {      // return annotation; no reply, so unit
    count = count + 1;        // + is numeric only
    print("tick " ++ count);
  }
}

var g = new Greeter("AIPL");
send g.tick();                // asynchronous; does not wait for a reply
send g.tick();
```

```
hello, AIPL
tick 1
tick 2
```

`init` is called automatically by `new Greeter("AIPL")`. `tick` does not `reply`, so it declares `unit` and the coverage check does not apply. `send` does not wait for a reply, so the two `tick`s are processed in order. `count + 1` is `int` with `int`, so it stays `int` and prints as `1`, `2`.

The inferred effects are `Greeter#init !{log}` and `Greeter#tick !{log, mut}`.

24 g2: `now` / `future` / `await` and annotations

```
// g2: now / future / await and the return annotation
class Calc {
  // with an annotation, every reply is matched against this type and
  // replying on every path is checked as well
  method twice(a) : int {
    reply(a * 2);
  }

  // without one, the type is inferred from reply (-> string)
  method label(a) {
    reply("v=" ++ a);
```

```
  }
}

var c = new Calc();

var s = now c.twice(21);      // now is an expression; the result is int
print("twice = " ++ (s + 1)); // usable as int in arithmetic

var f = future c.twice(10);   // future gives future int
var v = await f;              // await takes the value out
print("awaited = " ++ v);

print(now c.label(7));        // the inferred string
```

```
twice = 43
awaited = 20
v=7
```

`twice` declares `int`, so `now c.twice(21)` is `int` and can be used in the arithmetic `s + 1`. `label` omits the annotation, and `string` is inferred from `reply("v=" ++ a)`.

This sample writes no deadlines, so the type checker emits three warnings.

```
[warn] line 16, col 9: now without a deadline waits forever;
  write `now ... timeout <ms> else <expr>`
```

25 g3: several actors, and a `now` across actors

```
// g3: several actors, and a now that crosses actors
class Stock {
  var n = 10;

  method take(k) : int {
    n = n - k;
    if (n < 0) { n = 0; }
    reply(n);
  }
}

class Front {
  // Stock.take returns int; that becomes the type of the now,
  // which in turn feeds place's own string
  method place(k) : string {
    var left = now stock.take(k);
    if (left > 0) {
      reply("ok, left=" ++ left);
    } else {
      reply("sold out");
    }
  }
}

var stock = new Stock();
var front = new Front();

print(now front.place(3));
print(now front.place(99));
```

```
ok, left=7
sold out
```

`Stock.take` returns `int`; that becomes the type of the `now` inside `Front.place`, which then feeds `place`'s own `string`. Types propagate across actor boundaries.

Because `place` declares `string`, the checker verifies that both branches of the `if` reply. Delete one and you get `some execution path does not reply`.

Note that `n = n - k` with `int n` forces `k` to be `int`, consistent with `place(3)` at the call site; writing `place("x")` would be an *argument* type error.

Both methods have effect `{mut}` (assignment to field `n`), and `Front#place` inherits `Stock#take`'s `{mut}` through the `now`.

26 g4: `select`, and which message a `reply` answers

```
// g4: select / case / timeout, and which message a case's reply answers
class Worker {
  // the job message that arrives from outside
  method job(k) : string {
    reply("direct:" ++ k);
  }

  // wait for job with select. The reply in the case body answers job,
  // so serve itself replies to nothing and stays unit
  method serve() : unit {
    print("waiting");
    select {
      case job(k) -> {
        print("got " ++ k);
        reply("via select:" ++ k);
      }
      timeout 500 -> {
        print("timed out");
      }
    }
  }
}

var w = new Worker();
send w.serve();
print(now w.job(1));
```

```
direct:1
waiting
timed out
```

The `reply` in the body of `case job(k)` answers `job`, not `serve`, so `serve` may stay `unit`. The type checker also matches that `reply` against `job`'s return type.

The output shows **a design caveat this example carries**. Ordinary method dispatch processed `now w.job(1)` first, so by the time `serve`'s `select` ran, no `job` was left in the mailbox and it fell through to the `timeout`. It is safer to design the message names you `select` on so that they are not called directly.

27 g5: exposing to the outside makes annotations mandatory

```
// g5: exposing an actor makes the return annotation mandatory
//     web_listen opens the border, and POST /api/send can reach any actor.
//     The far side cannot see the body, so the reply type can only
//     travel as a declaration.
class Echo {
  method say(msg) : string {   // without the annotation this is a type error
    reply("echo: " ++ msg);
  }
  method note(msg) : unit {
    print("[note] " ++ msg);
  }
}

var echo = new Echo();

web_listen(8080);
web_expose("/echo", "echo");

print("try POST /api/x/echo with method=say&args=hi");
```

This program type-checks. Remove the annotation on `say` and you get:

```
[Type error] line 16, col 1: actor Echo is reachable from outside this
  program, but say has no return type annotation; a remote caller cannot
  see the body, so the reply type has to be declared (method say(...) : T)
```

Running `web_listen(8080)` starts the gateway and blocks listening, so check it from another terminal.

```
$ curl -s -X POST http://localhost:8080/api/x/echo \
       -d 'method=say&args=hi'
```

28 g6: effect annotations and propagation

```
// g6: the effect annotation !{...}
//
//     The existing capability classification, made static. Eight effects:
//       mut  write your own state        net  communication outside the node
//       time reading the clock, waiting  ai   model inference
//       io   input/output to a device    fs   persistence
//       mem  dynamic allocation          log  output for observation only
//
//     Omit the annotation and it is inferred from the body. Write it and
//     "the body's effects  the declared effects" is checked (more is allowed).
class Counter {
  var n = 0;

  method bump() : int !{mut} {   // assignment to a field is mut
    n = n + 1;
    reply(n);
  }
  method peek() : int !{} {      // effect-free: pure, so free to replicate
    reply(n);
  }
  method show() : unit !{log} {  // print is log
    print("n = " ++ n);
  }
}
```

```
class Planner {
  // ai_call leaves the box, so it has both ai and net.
  // It shows up in the signature, so which actors leave the box is visible
  method plan(goal) : string !{ai, net} {
    reply(ai_call("plan: " ++ goal));
  }
}

class ViaNow {
  // now waits, so it inherits the callee's effects.
  // Without declaring {ai, net} this is a type error
  method run(g) : string !{ai, net} {
    var r = now planner.plan(g);
    reply(r);
  }
}

class ViaSend {
  // send does not wait, so it does not inherit. Effect-free is fine
  method fire(g) : unit !{} {
    send planner.plan(g);
  }
}

var planner = new Planner();
var c = new Counter();
var a = new ViaNow();
var b = new ViaSend();

// ai_call would hit an external API, so only Counter is exercised here
print(now c.bump());
print(now c.peek());
send c.show();
```

```
1
1
n = 1
```

The effect table printed by the type checker is the point of this sample.

```
[method effects (inferred / declared)]
  Counter#bump !{mut}          <- assignment to a field
  Counter#peek !{}             <- pure
  Counter#show !{log}          <- print
  Planner#plan !{ai, net}      <- ai_call
  ViaNow#run   !{ai, net}      <- now, so it inherited plan's effects
  ViaSend#fire !{}             <- send, so it did not
```

Remove the `{ai, net}` declaration from `ViaNow.run` and it fails with `declares {} but its body has {ai, net}`. `ViaSend.fire` may stay `{}`. The rule that **only the side that waits carries the callee's effects** is visible right here.

29 g7: `now` / `await` with deadlines

```
// g7: now / await with a deadline
//
//     Without a deadline, a now or await waits forever if no reply comes.
//     An actor is one thread, so while it waits it cannot process any
```

```
//     other message -- it turns to stone.
//
//       now t.m(a) timeout <ms> else <expr>
//       await f    timeout <ms> else <expr>
//
//     The else branch must have the same type as the body.
class Slow {
  method work(k) : int !{time} {
    wait(300);              // takes 300 ms
    reply(k * 2);
  }
  method fast(k) : int !{} {
    reply(k * 2);
  }
}

var s = new Slow();

// returns in time
var a = now s.fast(21) timeout 1000 else 0;
print("fast = " ++ a);

// times out -> the value of the else branch
var b = now s.work(21) timeout 50 else 0;
print("slow(timed out) = " ++ b);

// awaiting a future with a deadline
var f = future s.fast(5);
var c = await f timeout 1000 else 0;
print("await = " ++ c);
```

```
fast = 42
slow(timed out) = 0
await = 10
```

`work` takes 300 ms, so the `now` with a 50 ms deadline times out and becomes the `0` of the `else` branch. `fast` makes the 1000 ms deadline and returns `42`.

This sample also passes under `AIOS_STRICT_DEADLINE=1` (every wait has a deadline). Change `else 0` to `else "zero"` and you get:

```
[Type error] now: the else branch has type string but the call returns int
```

30 g8: equality, unary minus, boolean literals

```
// g8: equality, unary minus, boolean literals
//
//     The lexer and the evaluator had all of these, but the expression
//     grammar did not, so they could not be written.
//
//       ==  !=          equality, on int / float / string / bool
//       - e             unary minus; lowers to neg(e)
//       true  false     boolean literals
//
//     Precedence is equality < comparison < ++ < additive < multiplicative
//     < unary minus, so 1 + 2 == 3 reads as (1 + 2) == 3.
class C {
  var flag = false;                     // a bool field
  method set(v) : unit !{mut} { flag = v; }
```

```
  method get() : bool !{} { reply(flag); }
  method t() : unit !{log, mut} {
    flag = true;
    if (flag == true)  { print("literal true ok"); }
    if (flag != false) { print("literal false ok"); }
    var b = 1 < 2;
    print("b = " ++ b);
    print("not-eq: " ++ (true != false));
  }
}
var c = new C();
send c.t();
```

```
literal true ok
literal false ok
b = true
not-eq: true
```

All of these existed in the lexer and the evaluator but had *no expression grammar rule*, so they could not be written; the previous edition listed them as a known gap. `flag` is initialized with `false`, so it is `bool`, and the parameter of `set(v)` is wired to `bool` as well.

31 g9: passing an actor as an argument

```
// g9: passing an actor as a message argument
//
//     With the parameter annotation `method m(x: D)` you can send to the
//     actor you received. Without it the parameter type is undetermined
//     and you get `send target is not actor`.
class D { method ping() : unit !{log} { print("pong"); } }
class P {
  method viaAnno(x: D) : unit !{log} {
    print("got actor");
    send x.ping();
    print("after send");
  }
}
var d = new D();
var p = new P();
send p.viaAnno(d);
```

```
got actor
after send
pong
```

The parameter annotation `d: Device` fixes the class of that argument; passing anything else is a type error. This annotation also adds an edge for obligation levels (§15), which is exactly why **levels did not need to be carried on actor types**.

32 g10: two classes with a field of the same name

```
// g10: two different classes may have a field of the same name
//
//      A class body is checked under that class's own fields, so
//      Fork's held and Latch's held are different things.
class Fork {
```

```
  var held = 0;
  method take() : int !{mut} { held = 1; reply(held); }
}
class Latch {
  var held = 0;
  method arm()  : int !{mut} { held = 2; reply(held); }
}
var f = new Fork();
var l = new Latch();
print("fork  = " ++ (now f.take() timeout 200 else -1));
print("latch = " ++ (now l.arm()  timeout 200 else -1));
```

```
fork  = 1
latch = 2
```

This is a regression test. The OCaml implementation used to *stack* fields into the type environment, so an assignment to the second class's `held` was rejected as an overloaded name (Python and JavaScript were correct). Of the 586 programs at hand, 119 contained two classes with a field of the same name, but in almost all of them another error came first and hid it.

33 Negative samples

`docs/samples/reply_inference/` holds samples whose purpose is to confirm that they *do not* pass.

File	What happens
`s2_missing_reply`	the result of a forgotten `reply` is used in arithmetic
`s3_conflict`	branches `reply` with different types
`s5_overload_ambiguity`	`a + b` with no principal type
`s8_annotation_conflict`	a `reply` that contradicts the declaration
`s9_missing_path`	some path does not `reply`
`s10_expose_unannotated`	an exposed actor without annotations
`s12_service_exposed`	the same, in the exposed integration sample
`s14_select_pattern`	a `case` arity that differs from the signature
`s15_double_reply`	a double `reply`
directly under `docs/samples/`	
`soundness2_effect_gap`	`future`+`await` escapes the effect check (a known hole)

Part VI
Building and running

34 Building the implementation

```
cd src && make thread_repl
```

Building with `dune` fails because `src/lexer.ml` is generated twice, so use `src/Makefile`.

35 Running a program

Pass a file of REPL commands with `-f`.

```
cat > run.repl <<'EOF'
load docs/samples/guide/g7_deadline.aipl
compile
```

```
EOF
./src/abclrepl_thread -q -f run.repl < /dev/null
```

The main REPL commands are `load` / `compile` / `list` / `send` / `ast` / `pprint` / `script` / `exit`.

36 Running only the type checker

The REPL depends on SDL, so a small driver exists for type checking alone.

```
ocamlfind ocamlc -package unix -thread -linkpkg -w -a -I src -o tc \
  src/location.cmo src/types.cmo src/ast.cmo src/typing_env.cmo \
  src/infer.cmo src/typecheck.cmo src/parser.cmo src/lexer.cmo \
  docs/samples/reply_inference/tc_main.ml

./tc docs/samples/guide/g3_actors.aipl
```

It prints the table of inferred and declared return types, the effects of each method, and the signature of each class.

37 The type/run-time differential test

There is a harness that compares the return type the checker assigned with the type of the value actually `reply`ed at run time.

```
python3 scripts/type_runtime_diff.py abclc/*.aipl docs/samples/guide/*.aipl
```

With `AIOS_TYPE_TRACE=1` the implementation prints `[rtype] Class#method = tag` on every `reply`. This mechanism actually found an inconsistency where integer arithmetic returned a `float` ($\vdash e$: `int` yet $e \Downarrow$ `VFloat`). A deliberately mismatching negative sample can be marked as known with `// TYPE_RUNTIME_DIFF: expect-mismatch`.

Run it whenever you change the types. It is the standing device for catching a break in preservation.

38 Environment variables

Variable	Effect
`AIOS_QUIET=1`	suppress debug output such as type schemes
`AIOS_STRICT_DEADLINE=1`	make a `now` / `await` without a deadline an error
`AIOS_LAX_OVERLOAD=1`	make an ambiguous overload a warning instead of an error
`AIOS_LAX_EXPOSE=1`	make a missing annotation on an exposed actor a warning
`AIOS_TYPE_TRACE=1`	print an `[rtype]` line on every `reply`
`AIOS_MODEL_PROVIDER`	choose the provider for AI calls
`ABCL_AI_PROVIDER`	the same (old name)
`REMOTE_REVIEWER_HOSTPORT`	the target of `remote_review`

39 Running on real hardware — Xinu / Raspberry Pi 3, Pi 4 and Pi 5

AIPL programs also run on **bare-metal Raspberry Pis** (Embedded Xinu): no operating system underneath, no libc, and the kernel itself carries the actor machinery. There are **three boards**. All of them accept the same canonical AIPL, but the machinery inside differs on each — which is much of what makes this section worth reading.

	Pi 3	Pi 4	Pi 5
Address	`192.168.3.50:8080`	`192.168.3.100:80`	`192.168.3.101:80`
Execution	a VM interprets `.avm`	AIPL→C→in-kernel `cc` JIT	
Front end lives	on the Mac	**on the board** (`abcl2c`)	
Actor vehicle	**real Xinu processes**		cooperative pump (one thread)
Ten guide samples	9 match	**all ten**	**all ten**

The Pi 4 sits **between the other two**: it executes like the Pi 5 (JIT) and schedules like the Pi 3 (real processes).

39.1 Three runtimes, and what each is for

Runtime	What it does	Checks
OCaml REPL (canonical)	checks types, effects, deadlines, the seven disciplines	**yes**
Host VM (Mac)	runs `.avm`; values are readable	no
Xinu hardware (Pi 3)	the in-kernel VM runs `.avm`	no
Xinu hardware (Pi 4)	lowered to C and JITed; actors are real processes	no
Xinu hardware (Pi 5)	AIPL is lowered to C and JITed to AArch64 in-kernel	no

This is the most important point in this section. **The `.avm` path does not type-check.** Return-type annotations (`: string`), obligation levels (`@3`) and effects (`!{log}`) are **accepted as syntax and then discarded**. Every static guarantee comes from the canonical (OCaml) side; the hardware is where **something already checked** is run. So the workflow has two stages: pass the type check of §36 first, then lower to `.avm` and ship it to the board.

39.2 Compiling to `.avm` and loading it

An `.avm` file is integer actor bytecode (the first four bytes are `AVM1`), and the format is **the same** on the host VM and on the Pi 3.

```
# 1. .aipl -> .avm
./_build/default/compile_avm.exe prog.aipl prog.avm

# 2. check the values on the host VM first
./_build/default/server.exe 8099 --no-open
curl --data-binary @prog.avm "http://127.0.0.1:8099/actor/loadvm?ask=0"
curl "http://127.0.0.1:8099/api/console"

# 3. send it to the board (the Pi 3 listens on 8080)
curl --data-binary @prog.avm "http://192.168.3.50:8080/actor/loadvm?ask=0"
loadvm: body=131 accepted=1 spawned actor id=1
```

No kernel reflash is needed. The `.avm` module is loaded into the running kernel and starts as an actor there and then. The SD card is only swapped when a **new VM instruction** has been added.

- **Without `?ask=0` nothing comes back.** Forget it and the request sits for 25 seconds and closes with zero bytes received. It looks exactly like a dead board, so suspect this first.
- The board is **fragile under back-to-back HTTP**; leave 2–4 seconds between requests.

39.3 Reading values back from the board

`print` on the Pi 3 goes to the serial line and the screen. **Since 2026-09-04 the recent output can also be read with `GET /api/console`**, in the same shape the host VM uses, `{"total":N,"lines":[...]}`; add `?clear=1` to empty it after reading.

```
curl --data-binary @g1_hello.avm "http://192.168.3.50:8080/actor/loadvm?ask=0"
loadvm: body=168 accepted=1 spawned actor id=1

curl "http://192.168.3.50:8080/api/console"
{"total":3,"lines":["a2: hello, AIPL","a2: tick 1","a2: tick 2"]}
```

Until this went in, g5 was the only sample whose output had been checked on the Pi 3 itself. The Pi 5 returns the output in the `POST /cc` reply, so the two boards could not be compared by running the same program on both. On the first day it was readable, two defects surfaced that had been invisible until then (a double `init` call, and actors from the previous program staying alive — §39.6). **What you cannot observe, you cannot call correct.**

To read a return value only, external exposure still works (§21).

```
// g5:
//     web_listen     POST /api/send
//
class Echo {
  method say(msg) : string {   //
    reply("echo: " ++ msg);
  }
  method note(msg) : unit {
    print("[note] " ++ msg);
  }
}

var echo = new Echo();

web_listen(8080);
web_expose("/echo", "echo");

print("POST /api/x/echo   method=say&args=hi      ");
```

```
curl "http://192.168.3.50:8080/api/routes"
/echo -> a6

curl "http://192.168.3.50:8080/api/x/echo?method=say&args=hi"
echo: hi
```

`args` is URL-decoded before it is passed in (`%20` and `+` become spaces). The reply is awaited for up to 90 seconds, and the wait ends as soon as the reply arrives.

39.4 An on-board model — `ai_call`

The Pi 3 kernel has **a small language model baked into it**. `ai_call` does not reach out to a service; it **runs inference on the board**.

Model	`stories260K` (Llama 2 shape; `dim`=64, 5 layers, 8 heads, vocab 512)
Size	about 260K parameters, 1.06 MB in fp32
Where	the `.rodata` of the kernel image (kernel: 1.09 MB → 2.16 MB)
Arithmetic	soft float, D-cache off, 32-bit A53
Measured	2–6 seconds per `ai_call` round trip (24 tokens generated)

```
curl "http://192.168.3.50:8080/api/x/sage?method=ask&args=Once%20upon%20a%20time"
MODEL[, there was a little girl named Lily. She loved to play outside in the p]END
```

The effect table (§22) gives `ai_call` the effect `{ai, net}`, yet on the Pi 3 **nothing leaves the board**. The effect checker over-approximates, so the fact that `{net}` does not actually occur here does not compromise its soundness: it may forbid too much, never too little.

The host VM (Mac) has no model. Calling `ai_call` there returns a string prefixed with `(no local model on host)`. It does not manufacture a plausible-looking answer — the absence of a model is written into the value rather than quietly hidden.

39.5 What the hardware path does not have

The `.avm` path accepts a **subset** of the canonical language. What is missing **fails on the spot**; nothing is dropped silently.

Feature	`.avm` path	Note
Classes, fields, `send` / `send!`	yes	`send!` is treated as `send`
Method-local `var`, `reply`	yes	locals become hidden fields
`if` / `while`	yes	parentheses optional
`now` / `future` / `await` / deadlines	yes	via **continuation splitting** (below)
`select` / `case` / `timeout`	yes	a `case` name must be a real method
Strings (values, `++`, arguments)	yes	
`web_listen` / `web_expose`	yes	
`ai_call`	yes	on-board model (§39.4)
Type / effect / level annotations	accepted, **discarded**	checked on the canonical side
`float` and float literals	no	values are integers and strings only
Arrays (`array_*`)	no	`avm: unsupported expression`
Math builtins (`sqrt` etc.)	no	likewise
`call` / calling one's own method	no	the VM has no call instruction
Deadline without `else` (`result`)	yes	failure is a reserved value; no new instruction
`is_ok` / `value`	yes	lowered to a comparison and a branch
`replyto` / `answer` / parameter `reply`	yes	a reply destination is an actor id (`__snd`)
`acquire` / `release`	yes	new instructions `0x53`/`0x54`; named locks
`spawn`	no	likewise
`remote(...)` sends	yes	instructions `0x55`/`0x56`; carried over UDP/9010
`await` in expression position	no	only `var x = await f;` (continuation splitting)

A word on **continuation splitting**. The VM has no instruction for a synchronous call that returns a value. So wherever `now`, `await` or `select` appears, the method is **cut in two** and the remainder is emitted as a separate method. To carry values across the cut, method-local `var`s are turned into **hidden fields of the actor**. The destination of `reply` needs the same care: after a split, `sender` is no longer the original caller, so the caller is saved into `__snd` on entry to the method. The behaviour a user sees matches the canonical implementation, but **more methods are generated** than were written.

39.6 The Pi 3 holds exactly one module at a time

The Pi 3 VM has **a single module slot** (`static unsigned char vm_mod[...]`). `/actor/loadvm` **overwrites it**, so **only the most recently loaded module is live**.

This bit silently for a long time. Routes registered earlier stayed in the table; only the actors behind them stopped working — calling one got no reply, and `vm_web_call` sat out its full 90 seconds and returned empty. After loading thirteen modules back to back, the `/sage` and `/echo` exposed first had both gone quiet, while an actor loaded at that moment answered instantly.

On 2026-09-04 this was brought in line with the Pi 5. A `POST /cc` replaces the Pi 5's whole actor world; `/actor/loadvm` now likewise **folds away the previous module's actors** and empties the route table first (resident C actors are left alone). Without that, the old actors stayed alive as Xinu threads and **ate the process table** — running the ten guide samples in order, the ninth (g9) stopped starting its actors and HTTP wedged shortly after (measured). It also removes the stale method-name pointers that the mailboxes still held into the replaced `vm_mod`.

- **Do not use `kill()` to fold them.** An actor thread calls `alloc_obj` inside the `spawn` opcode and holds `objects_mu` there; killing it mid-way stops every later spawn, taking `webactor` and HTTP down with it (also measured). **Mark them and wake them** instead, and let each leave its own loop.
- Exposed routes **live and die with the program**: after a load, `/api/routes` lists only the new one.
- **Try one module at a time** on the hardware; check it before loading the next.

39.7 The ten canonical guide samples on hardware

Every one of g1–g10 from Part V lowers to `.avm`, loads onto the Pi 3, and starts as an actor. **On 2026-09-04 all ten were checked on the hardware down to their output** (§39.3). Before that only `accepted=1` — "loaded and started" — had been observed, plus the output of the exposed samples g5 and `/sage`.

	Canonical	Pi 3 hardware
g1	`hello, AIPL / tick 1 / tick 2`	matches
g2	`twice = 43 / awaited = 20 / v=7`	matches
g3	`ok, left=7 / sold out`	matches
g4	`waiting / got 1 / via select:1`	matches
g5	`/api/x/echo` → `echo: hi`	matches
g6	`1 / 1 / n = 1`	matches
g7	`fast = 42 / slow(timed out) = 0 / await = 10`	matches
g8	`... / b = true / not-eq: true`	matches
g9	`got actor / after send / pong`	**order differs** (below)
g10	`fork = 1 / latch = 2`	matches

The one mismatch is not a gap in the front end; it comes from the shape of the execution vehicle. (g8's `b = true` was brought into line on both boards on 2026-09-04, §39.8.)

- **g9's ordering** — Pi 3 actors are **real processes**, so the target of `send x.ping()` can run before `print("after send")` (the board printed `got actor / pong / after send`). The Pi 5 is a cooperative pump, so it always produces the canonical order. **Both are correct AIPL**: `send` promises no ordering.

That comparison holds **for one module loaded at a time**; it does **not** mean the ten run side by side (§39.6).

39.8 The Pi 5 — the other board

The Pi 5 (`192.168.3.101`) **works differently**. Instead of interpreting bytecode, it lowers AIPL to C on the board and the in-kernel C compiler `cc` **JITs that to AArch64 machine code**.

	Pi 3	Pi 4	Pi 5
Execution	a VM interprets `.avm`	AIPL→C→in-kernel `cc` JIT	
Front end	`compile_avm` (Mac)	`abcl2c` on the board (same source)	
Submission	`POST /actor/loadvm?ask=0`	`POST /cc`	
Port	8080	80	80
Reading output	`/api/console`	in the reply	
Reading return values	`web_expose` + `/api/x/` (all three)		
Programs resident	one (a new one replaces it)		
Actor vehicle	**real Xinu processes**		cooperative pump (one thread)
`select`	continuation split + interception (all three)		
`future` / `await`	continuation split	runs at once	(none concurrent)
`wait(ms)`	a VM opcode	on the app worker	on a dedicated process
Booleans	a tagged value; printed `true` / `false` (§39.8)		
Value kinds	integers and strings	int, string, float, list	
`ai_call`	opcode `0x52`	runtime `v_ai_call`	
measured	2–6 s	0.79 s	**0.42 s**
Ten guide samples	**9 match**	**all ten**	**all ten**
Kernel	1.09→2.16 MB	2.02→2.07 MB	0.94→2.05 MB

Running a program

Because the front end **lives on the board**, you can post AIPL source directly. If the first word is `class` it goes through `abcl2c`; otherwise it is treated as C. **Whitespace and comments are skipped before that test**, so canonical sources that open with `//` pass through unchanged.

```
curl --data-binary @g1_hello.aipl "http://192.168.3.101/cc"
hello, AIPL
tick 1
tick 2
=> 0
```

The trailing `=> 0` is `main`'s return value. From the shell, `cc <file>` / `make <file>` handle `.abcl` / `.aipl` transparently. The in-kernel model can also be poked directly with `llm <prompt>` (output goes to the serial line).

What the program becomes

What you write stays AIPL, but the generated C is readable. Classes become **numbers**, objects are elements of `g_obj[]`, methods become functions named `m_<Class>_<method>`, and a method call becomes a numeric `dispatch`.

```
struct Obj { int cls; int f[1]; };   /* f[] holds the fields */
struct Obj g_obj[2048];
int v_stock;                          /* top-level vars become globals */
int m_Stock_take(int self, int a0, int a1, int a2, int a3) { ... }
int __new_init(int id, int meth, int a0, int a1, int a2, int a3);
int __dl_call(int to, int meth, int a0, int a1, int a2, int a3, int ms, int elsev);
int dispatch(int self, int meth, int a0, int a1, int a2, int a3);
int main() { v_stock = v_int(g_spawn(0)); ... }
```

In this `cc`, `int` is 8 bytes (stated in `cc/ccpriv.h`). That matches the width of AIPL's tagged 64-bit values, so a string (a pointer into the heap) survives being held in an `int`. Note that **a function takes at most 8 parameters**; a ninth gives `cc: too many parameters (max 8)`.

Values go through runtime helpers — `v_int` `v_str` `v_floatlit` `v_add` `v_sub` `v_mul` `v_div` `v_lt` `v_le` `v_eq` `v_ne` `v_and` `v_or` `v_not` `v_truthy` `v_int_of` `v_print` `v_ai_call` `v_list_*`. `v_add` concatenates when either side is a string, so `++` is mapped onto it.

The in-kernel model

`ai_call` lowers to `v_ai_call`, which renders the prompt to a string, calls `llm_run`, copies the result into the value heap and returns it as a string. The model is **the same one as the Pi 3** (§39.4): `ai_call("Once upon a time")` agrees to the byte (72 of them) across the Pi 3, the Pi 5 and the reference build on the Mac. Only the speed differs: **the Pi 5 takes 0.42 seconds per call**. The Pi 3 takes 2–6 seconds when freshly booted, and took 16.6 seconds once thirteen modules had been loaded and actors had piled up (generation yields to the scheduler per token, so more resident actors means slower — though **that causal link has not been isolated**).

What the on-board front end (`abcl2c`) accepts

Accepted	classes, fields, methods, `var`, assignment, `if`/`else`, `while` (`do` optional), `return`
	`new` (arguments are passed to `init` when there is one), `send`, `now` (returns a value)
	`reply(e)` — the same as `return e` here, since `now o.m()` takes `dispatch`'s return value. It must be **the last statement of its block**; otherwise it is refused, because a `return` would change the meaning
	annotations `: T` / legacy `-> T` / `!{log, mut}` / `@ 2` / `x: D` / `var n: int` — **accepted and discarded** (checking is the canonical side's job)
	`++`, **`true` / `false`** (`bool` is an `int` here, so 1 / 0), unary minus
	postfix deadlines: `now t.m(a) timeout <ms> else <expr>` (see below)
	`web_listen` / `web_expose` (see below)
	`wait(ms)` — only on the shell `cc`/make path (see below)
	`print` / `puts`, `ai_call`
	`send!` (treated as `send`), **`call g(args);`** and bare `g(args);` (calling one's own method)
	Floats — literals and `float` fields, passed to the value runtime's `v_floatlit` as an IEEE-754 bit pattern
	Arrays `array_empty` / `push` / `get` / `set` / `len` / `zeros` — mapped onto the runtime's lists
	Maths builtins `sqrt exp log log10 sin cos tan asin acos atan floor ceil round abs neg` — written by hand, since there is no `libm` (`sqrt` is the `fsqrt` instruction; the rest reduce the range and then use a series, §39.8)
	`result` — a deadline written without `else` (`now o.m() timeout N`, `await f timeout N`) plus `is_ok(r)` / `value(r, default)`
	`future` / `await` — but **not concurrent**; they run on the spot (§39.8)
	`select` / `case` / `timeout` — but they **wait, on the Pi 3's continuation scheme** (§39.8)
	`replyto` / `answer` / parameter `reply` — since `reply(v)` is the return value here, one reply slot is taken per `now` to carry a destination as a value
	`acquire` / `release` — pairing and order are checked canonically; what is kept here is a runtime watch only
	`remote("host:port","actor")` — send to, or call, an actor another node published. UDP/9010; the datagram is shared by all three boards and the Mac host VM
	a field initialiser may be `new K()` (the canonical way to hold an actor is in a field, so without this you get `send target is not actor`)

Refused	`spawn`

Refusals name what was refused:

```
not supported on this device: select
unknown function: sqrt
new: unknown class: Zzz
reply must be the last statement of its block on this device
```

Nothing quietly turns into something else.

The ten canonical samples on the board

All ten of g1–g10 now run unchanged when posted to `POST /cc`. Previously none of them did — every one died on its return-type annotation. But **"runs" and "matches the canonical semantics" are not the same thing**.

Sample	Output (`POST /cc`)	Canonical?
g1	`hello, AIPL` / `tick 1` / `tick 2`	yes
g2	`twice = 43` / `awaited = 20` / `v=7`	yes
g3	`ok, left=7` / `sold out`	yes
g5	exposed, then `echo: hi` from `/api/x/echo`	yes
g6	`1` / `1` / `n = 1`	yes
g8	`literal true ok` / `literal false ok` / `b = true`	yes
g9	`got actor` / `after send` / `pong`	yes
g10	`fork = 1` / `latch = 2`	yes
g7	`fast = 42` / `slow(timed out) = 0` / `await = 10`	yes
g4	`waiting` / `got 1` / `via select:1`	yes

On 2026-09-04 all ten came to match over `POST /cc` alone. The last two holdouts were g4 (`select` did not wait, §39.8) and g7 (`wait` was unavailable under `/cc`, §39.8); both were **properties of the path**, not gaps in the front end. **There is no longer any need to pick a path.**

g8's `b = true` was brought into line on 2026-09-04 (§39.8). Until then both boards held `bool` as an `int` and printed `b = 1`.

Deadlines are checked afterwards

`now t.m(a) timeout <ms> else <expr>` is accepted, but it **does not interrupt the call**. Execution here is a cooperative pump and `dispatch` runs to completion, so the deadline can only be used as an **after-the-fact test of whether the call overran**.

```
now s.quick(7) timeout 1000 else -99   ->   7     (within the deadline)
now s.heavy(7) timeout    1 else -99   -> -99     (overran -> the else branch)
```

The value agrees with the canonical semantics, but the call itself still runs to the end. A program that relies on a deadline to *stay responsive* does not get that here.

Exposing actors — `/api/x/`

`web_expose("/echo", "echo")` makes an actor reachable at `GET /api/x/<path>?method=<name>&args=<text>`.

```
curl --data-binary @g5_web.aipl "http://192.168.3.101/cc"
curl "http://192.168.3.101/api/x/"
  exposed routes (web_expose):
    /api/x/echo  -> actor #0
curl "http://192.168.3.101/api/x/echo?method=say&args=hi"
  => echo: hi
```

- **The port argument is ignored.** The board's HTTP server already listens on 80, which satisfies what `web_listen` is for (opening the boundary). The board records the number and serves the exposed routes on 80.

- **Exposure lives and dies with the program.** Loading the next program clears the routes. The Pi 3 failure mode — **stale routes that survive while their target dies** (§39.6) — does not happen here.
- The second argument is the name of a top-level `var`. The board identifies actors by number, so the name is resolved at translation time. An unknown name is refused by name.

The runaway budget, and `wait`

JITed code is **cut off on a wall clock** in case it runs away, and **the budget depends on the path**.

Path	Compute budget	`wait(ms)`
`POST /cc`	**5 s**	**works** (10 s total)
shell `cc` / `make`	**10 s**	**works** (same)
`/actor/load`	250 ms	refused

Since 2026-09-04, `wait` works under `/cc` too, because both now run in **their own process on their own stack**: a runaway can only blow that stack and get cut off, never freeze the board.

This was not done by sleeping in place. HTTP handling can be entered from the network process *or* from the window-manager loop (`NULLPROC`), and `proc_sleep_us` on `NULLPROC` context-switches **to itself** when nothing else is runnable — so it does not sleep at all. Instead `/cc` adopts the shell's shape: spawn a runner and spin `proc_resched()` until it is gone.

Sleeping is not charged to the budget. The budget is there to catch **runaway computation**, and letting `wait(300)` eat it would defeat that. The **total sleep** per run is capped at 10 s instead.

```
curl --data-binary @g7_deadline.aipl "http://192.168.3.101/cc"
  fast = 42
  slow(timed out) = 0        <- wait(300) took effect. 0.40 s
  await = 10
```

There is a trap here that half-kills the board. The only thing advancing the runner is **the caller spinning `proc_resched()`**, and `proc_preempt()` never switches away from `NULLPROC`. So a runner that is given up on **never runs again**: it keeps the static stack, and every later `cc` answers "the previous run has not finished" (measured on hardware). Three guards close it: cap the total sleep below the 15 s give-up, mark an abandoned runner "return at once", and let the next call spin for 3 s to clear it.

`future` / `await` are not concurrent

`future o.m(x)` **runs on the spot**: it calls `dispatch` immediately, stores the value in a slot and returns a handle. `await h` reads that slot. A deadline on `await` **can never fire**, because the value is already there. In other words, **there is no "fire it off and collect it later"**.

The Pi 3 is the same in this respect — continuation splitting does not run anything concurrently either — so **the two boards have the same temperament here**.

A genuinely concurrent implementation was written and then removed, because it froze the board. It gave each call its own Xinu process and had `await` wait for it. The value heaps (`vheap_alloc`, `lheap_alloc`) were protected by masking interrupts, and that was not enough: the runaway state `g_deadline` / `g_aborted`, the output capture `g_cap` and the single FIFO behind `cc_pump()` were all still shared. The safeguards in place (a wall-clock cap, a liveness check before reusing a slot) **never fired** — something lower down stops first. **"Protect the allocator and you can go concurrent" was wrong.**

The maths builtins are written by hand

There is no `libm` on bare metal, so the canonical `Core.Math` fifteen live inside the value runtime. `sqrt` is AArch64's `fsqrt` instruction; the rest reduce the range and then evaluate a series.

Accuracy was checked against `libm` on the Mac before shipping. The first version was off by 2.4×10^{-4} for `sin` and by 10^{-2} for `atan`/`asin`/`acos`: the range reduction was too weak and the series had not converged. Folding `sin`/`cos` to multiples of $\pi/2$ and picking by quadrant, and shrinking `atan` below $\tan(\pi/12)$, fixed it.

Builtin	Worst relative error vs. `libm`
`sqrt` / `floor` / `ceil` / `round`	**0** (exact)
`log` / `log10` / `atan` / `asin`	$\sim 10^{-16}$
`exp` / `tan` / `acos`	$\sim 10^{-15}$
`sin` / `cos`	$\sim 10^{-11}$

`abs` and `neg` return an integer when given one (the canonical implementation only accepts `float`, so the hardware is the laxer of the two).

`result` — a deadline without `else`

Writing `now o.m() timeout N` or `await f timeout N` without an `else` yields `result`$\langle\tau\rangle$. On the hardware this is **the value itself on success** and **one reserved value on failure** — success is not wrapped, so nothing is allocated. `is_ok(r)` and `value(r, default)` are the only observers, which is all that is needed.

```
var r = now a.f(21) timeout 1000;
print("is_ok = " ++ is_ok(r));      -- is_ok = true
print("value = " ++ value(r, 0));   -- value = 42
```

The failure value prints as `err` and is false as a condition.

Booleans are a tagged value

The canonical implementation prints `print("b = " ++ b)` as `b = true`. Both boards printed `b = 1` for a long time: booleans were plain 0/1, and the value carried nothing that would print them back as `true` / `false`. **Both were fixed together on 2026-09-04** — fixing one alone would have re-opened a difference we had just closed.

The scheme is the same tagged-value trick used for strings; only where the tag lives differs.

	Pi 3 (`.avm` VM, 32-bit)	Pi 5 (JIT, 64-bit)
Tag	`0x20000000` (bit 29)	bit 62 (low bit 0)
Value	the low bit	bit 1
Strings	`0x40000000` (bit 30)	a real address ($< 2^{48}$)

On the Pi 5 the tag cannot live on the integer side. `v_int(-1)` has every high bit set, so a high-bit tag would **turn negative numbers into booleans**. It goes in the "low bit 0, i.e. not an integer" space instead (floats use bit 60, lists bit 61).

Unlike strings, a boolean's tag must also be read when testing a condition. `false` carries the tag, so it is **not zero**, and a plain branch-if-zero would not see it as false. The Pi 3 routes `JZ` (`0x31`) through `vm_falsy`; on the Pi 5 `v_truthy` checks for a boolean first. Comparisons and logical operators (`< <= > >= == != && || !`) now return booleans.

In arithmetic and comparison they still behave as 0 / 1. Measured on both boards:

```
neg = -7        cmp = true      sum = -4      fld = true
branch ok       field-bool branch ok
```

One path still prints `1`: the Mac-side `--xinu-jit` (`POST /actor/load`) collapses `true` / `false` to `Int 1` / `Int 0` in its front end. Fixing that means adding a boolean to the AST, which reaches into type inference.

`select` — rebuilt on the Pi 3's continuation scheme

On 2026-09-03–04 this was rebuilt to match the Pi 3. Until then the Pi 5's `select` **did not wait**: there is no way to wait in a cooperative pump, so it merely scanned the messages already queued for the actor. In g4, `job` has not arrived when `send w.serve()` runs `serve`, so it **always fell into the timeout branch** — the gap this chapter used to call "the one place where the Pi 3 is ahead".

The Pi 3's scheme **waits without waiting**. It touches neither the scheduler nor the execution vehicle.

- The method containing the `select` sets a hidden field `__sel` to that `select`'s number and **ends there**.
- `case job(k) -> { ... }` is spliced in **at the top of method `job`**: if `__sel` holds its number, clear the mark and run the `case` body followed by the rest of the `select`'s method; otherwise run `job`'s own body.
- A `case`'s parameter maps **by position** onto the receiving method's parameter.

The message a `select` waits for therefore **arrives as an ordinary method call**. No suspension, no mailbox scan.

The `timeout` branch is decided by "nobody came", not by time. This device has no vehicle that can wait, so the branch runs once the FIFO has been drained and no one claimed the `select` (`__sel_sweep`); the millisecond value is unusable. The Pi 3 has a separate `__Timer` actor do `wait(ms)`, so **there it really is time** — but the meaning, "run the timeout branch if the awaited message never comes", is the same.

One more change: **the FIFO is now drained right after a top-level `send`**. Pi 3 actors are real processes, so the target runs first; the Pi 5 only queued the message, and a following `now` entered the target's method ahead of it, defeating the interception.

	Pi 3	Pi 5
How	continuation split + interception	**same**
Can it wait?	yes	**yes**
`timeout` decided by	a `wait(ms)` in another actor	the FIFO draining with no claim
g4's output	canonical	**canonical**

```
curl --data-binary @g4_select.aipl "http://192.168.3.101/cc"
waiting
got 1
via select:1            <- used to be "timed out"

# when nobody sends the awaited message
waiting
timed out
```

39.9 `remote` — calling an actor on another board

Canonically a send target is either a variable holding an actor reference or `remote("host:port", "actorName")` (see the table in §13). The second form now works on all three boards and on the Mac host VM. What it names is an actor published with `web_expose`.

```
class Echo { method say(x) : int !{} { reply(x * 2); } }
var e = new Echo();
web_expose("/echo", "e");

var d = now remote("192.168.3.101", "echo").say(21) timeout 2000 else -1;
send remote("192.168.3.50", "echo").say(5);
```

The datagram

All four implementations speak the same language: UDP port 9010, one line of readable ASCII (so `tcpdump` and `nc` show it as it is).

```
Q <reqid> <actor> <method> <arg...>
R <reqid> <value>
```

- What travels is the method **name**, not its number: numbers differ per board, so the receiving board resolves the name with its own `__method_id`.
- One argument. `<arg...>` runs to the end of the line, so it may contain spaces. A string of digits arrives **as an integer** on the other side (the canonical `now remote(...).step(3)` takes one).
- What comes back is the value re-read from the text the other side rendered. **No type is carried** — the promise holds because the canonical type checker has already matched the types on both sides.
- A deadline, an unknown host and an unknown method are **all** failures. With `else` you get that value; without it you get `err` (`result`). The timeout case stays in one place.

Retransmission, and at-most-once

UDP drops packets, so the request is re-sent every 200 ms. While it was sent once and then waited for, a board whose wait loop turns only a few times a second would **always** time out.

Retransmitting naively runs the remote method twice, so the receiver keeps the last answer per `(sender IP, reqid)` and, for a repeat of the same request, **returns the stored answer without running anything**. That makes it **at most once**, which is safe even for a method with side effects.

How each board carries it

Board	Carriage
Pi 3	the Xinu UDP device layer (`udpAlloc` / `open` / `read` / `write`); the receiving end is a resident process. VM instructions `0x55` / `0x56`
Pi 4 / Pi 5	their TCP is listen-only (passive open), so the frames are built by hand
Mac	OCaml Unix sockets. It is a **fourth peer**, which is what lets you flash one board at a time and still have something to talk to

The Pi 4 and the Pi 5 **never resolve the destination MAC with ARP**. The first request goes out with a broadcast destination MAC and a unicast destination IP, and **the decision to take it is made by the receiver**. The reply goes back to the requesting frame's sender, so no ARP is needed, and the MAC learned there is remembered — later requests are unicast. That breaks less than another ARP state machine, and is enough for a handful of boards.

Registering the callee (this differs per board)

The actor a call names must be **resident** on the board.

Pi 4	post it with `POST /cc?resident=1`
Pi 5	a plain `POST /cc` is enough
Pi 3	whatever was sent to `/actor/loadvm?ask=0` stays

A plain `POST /cc` on the Pi 4 runs the program and is done, so **nothing is left for an outside caller to reach** and the caller gets `err`.

Measured

That `say(21)` answers `42` was checked for **every pair** of the four implementations.

Caller →	Pi 3	Pi 4	Pi 5	Mac	unreachable
Pi 3	—	42	42	42	`else`
Pi 4	42	—	42	42	`else`
Pi 5	42	42	—	42	`else`
Mac	42	42	42	—	—

39.10 Traps common to all three boards (implementation side)

These are invisible to someone writing AIPL, but they are the holes we fell into **on two or more boards**. Putting actors on bare metal seems to trip in the same places every time.

- **Never take an actor thread down with `kill()`.** An actor can be inside an **interrupt-masked region** (allocating from the value heap, posting to a mailbox) or **holding a lock**. Killing it there leaves interrupts masked or the lock held: the board dies outright, or everything that waits on that lock spins forever. The right way is to **mark it and wake it**, and let it leave its own loop. **We hit this independently on the Pi 3 and the Pi 4.** The prerequisite is putting the "you are dead" flag into the *waiting* loop's condition — without that, killing looks like the only option.

- **Never leave a pointer into code you freed.** When one program replaces another, a function pointer left over from the previous run points into freed memory; the next allocation hands that memory out again, so the stale pointer **writes into someone else's buffer**. On the Pi 4 this surfaced as **a single corrupted byte**: a fixed offset of the next translation turned `'i'` into `'j'`, giving either `cc: undefined variable` or an instant board death. **Because the symptom depends on where the byte lands, it looks like a different bug every time.**

- **Never yield to actors while holding the value-heap lock.** An actor takes the same lock before its handler, so whatever you yielded to spins. **If the lock spins and "steals" after a limit, small examples happen to work** — the nastiest shape a bug can take.

- **If you put request/response on UDP, retransmission is part of the design.** Sent once and waited for, a request will always time out as soon as the receive side lags even slightly. On bare metal this looks like a broken receive path, and I rebuilt that path **twice**, each time only thinning the symptom. **The test that settles it is to have the peer answer repeatedly** — if that works, what is broken is not the receive path but the missing retransmission. And once you retransmit, keep a `(sender, reqid)` record on the receiving side so it stays **at most once**.

- **"I poked it while it was waiting and got an answer" does not prove the receive path is live.** It can be a late answer that your own wait window happened to catch. **Look at the time, not at whether it came.** Make the peer log when it sends, and keep two separate counts on the board: how many answers arrived, and how many were still being waited for. Misreading this cost a round trip before the real cause (the receive window in §39.9) came out.

- **After flashing, an HTTP answer is not proof that the board rebooted.** Until the power is cycled the board keeps answering from the old kernel, and polling cannot tell the difference. The symptom is "the thing I just fixed is not there", which sends you back to doubt the code. All three boards now report their build from `GET /version` (`kversion.c` is rebuilt every time through a `FORCE` dependency — without that `__DATE__` freezes and the version lies).

- **Do not guess at a cause after the board dies; make the failure observable first.** A dead board yields no information and every experiment costs a power cycle. **One flash spent on instruments beats five spent guessing.** On the Pi 4 the decisive step was adding a "translate only" stage (`?stage=xlat`), which tells you whether the board is healthy without running anything.

39.11 The Pi 4 — the third board

The Pi 4 (`192.168.3.100`) sits **between the other two**: it executes like the Pi 5 (AIPL lowered to C on the board and JITed) but its **actors are real Xinu processes** like the Pi 3's (`system/actorproc.c`). The on-board front end `abcl2c` is **the same source** as the Pi 5's, and for all ten canonical guide samples its output is **byte-identical** to the host translation.

Running a program

As on the Pi 5, you post AIPL source directly.

```
curl --data-binary @g1_hello.aipl "http://192.168.3.100/cc"
hello, AIPL
tick 1
tick 2
=> 0
```

`/cc` has **stages**. They let you check the front end without running anything, which is where to start when touching the hardware.

`POST /cc`	translate and JIT (default)
`POST /cc?stage=xlat`	**return the translated C; run nothing**
`POST /cc?resident=1`	resident load (reachable later via `/api/x/`)
`POST /compile`	treat the body as plain C and JIT it
`POST /actor/load`	resident load of Mac-side `--xinu-jit` C

Health is measurable with `?stage=xlat`: compare the board's translation with the host's, byte for byte. It is the one window that does not risk the board.

```
cc -DABCL2C_HOST_TEST -w -o /tmp/a2c cc/abcl2c.c
/tmp/a2c g1_hello.aipl > /tmp/host.c
curl --data-binary @g1_hello.aipl "http://192.168.3.100/cc?stage=xlat" > /tmp/board.c
diff /tmp/host.c /tmp/board.c        # any difference means the board is damaged
```

The ten canonical guide samples on hardware

All ten behave canonically (raw AIPL to `POST /cc`, run back to back).

Sample	Output on the hardware (`POST /cc`)
g1	`hello, AIPL` / `tick 1` / `tick 2`
g2	`twice = 43` / `awaited = 20` / `v=7`
g3	`ok, left=7` / `sold out`
g4	`waiting` / `got 1` / `via select:1`
g5	exposed, then `echo: hi` from `/api/x/echo`
g6	`1` / `1` / `n = 1`
g7	`fast = 42` / `slow(timed out) = 0` / `await = 10`
g8	`literal true ok` / `literal false ok` / `b = true` / `not-eq: true`
g9	`got actor` / `after send` / `pong`
g10	`fork = 1` / `latch = 2`

The translation is still byte-identical to the host's after the whole run. Regressions pass too: plain C through `/compile`, `/actor/load`, `ai_call` producing the same reference text as the Pi 3 and Pi 5 (**0.79 s**), and `/smp-bench` at 3.00× on four cores.

`wait` simply works

On the Pi 5, `/cc` had to be moved onto a dedicated process before `wait` could sleep. **The Pi 4 was already built that way**: HTTP is served by a **dedicated app worker**, with an **AIPL/LLM runtime process** behind it (`net_proc_main` / `app_proc_main` / `aipl_proc_main` in `loader/main.c`). So `wait(300)` is a real sleep and g7 matches at `slow(timed out) = 0` — no need to pick a path.

What to read when it stalls

The Pi 4 carries a full set of instruments. **Read them while the board is still alive.**

`GET /fault`	the last CPU fault (ESR / FAR / ELR / SPSR / SP)
`GET /api/aptab`	the raw actor table (pid, process state, queue, dead)
`GET /api/aprun`	`ap_run` give-ups, forced kills, worker replacements
`GET /api/mem`	free total, largest block, fragment count
`GET /jitstats`	`spawn_fails`, dropped messages
`GET /ticks`	stack canary (`stkbad`), context-switch count
`GET /wx`	whether W^X is enforced (it probes by writing)
`GET /reboot`	**watchdog reset** — no need to touch the power

`/reboot` only works while the board is alive. Use it before you run out of board.

What being real processes changes

- **g9's ordering can vary between runs.** The target of `send x.ping()` may run before `print("after send")`. The Pi 3 shares this property. **Both are correct AIPL**: `send` promises no ordering.
- After a top-level `send`, the board **lets the target run to quiescence** before continuing, to match the Pi 5's cooperative ordering. Without that, `select`'s interception does not fire.

Bad input does not stop the board

The front end has been swept with **mutation fuzzing** (ASan + UBSan, 4,000 cases). On the board, the eight inputs that used to freeze the kernel (`future` / `select` / `replyto` / `ai_cal` (a typo) / an argument-carrying `new` on a class with no `init` / `new` of an unknown class / a statement after `reply` / a deadline) were all replayed: **every one returns a named error at once, and the board survives every time**.

`new <unknown class>` is worth singling out. `class_idx()` returns −1 for a name it does not know, and that −1 used to go straight into the syntax tree, so the emitter read `CLS[-1]` — **an out-of-bounds read on real hardware**. The fuzzer found it.

Part VII
Appendix

40 Grammar summary

```
program    ::= decl+

decl       ::= "class" ID "{" field* method+ "}"
             | "var" ID "=" expr ";"
             | "var" ID "=" "new" ID "(" args ")" ";"
             | "send"  target "." ID "(" args ")" ";"
             | "send!" target "." ID "(" args ")" ";"
```

```
             | ID "(" args ")" ";"

field      ::= "var"   ID "=" expr ";"
             | "float" ID "=" expr ";"

method     ::= "method" ID "(" params ")" ret? lvl? eff? "{" stmt+ "}"
params     ::= (param ("," param)*)?
param      ::= ID | ID ":" ClassName | ID ":" "reply"
ret        ::= ":" ("int"|"float"|"string"|"bool"|"unit"|"any"|ClassName)
lvl        ::= "@" INT                    -- obligation level; inferred if omitted
eff        ::= "!" "{" (ID ("," ID)*)? "}"

target     ::= ID
             | "remote" "(" STRING "," STRING ")"

stmt       ::= ID "=" expr ";"
             | "var" ID "=" expr ";"
             | "var" ID "=" "new" ID "(" args ")" ";"
             | ID "(" args ")" ";"
             | "call" ID "(" args ")" ";"
             | "send"  ("self"|"sender"|target) "." ID "(" args ")" ";"
             | "send!" target "." ID "(" args ")" ";"
             | "if" "(" expr ")" stmt ("else" stmt)?
             | "while" expr "do" stmt
             | "{" stmt* "}"
             | "select" "{" case* timeout_case? "}"

case         ::= "case" ID "(" ids? ")" "->" "{" stmt+ "}"
timeout_case ::= "timeout" INT "->" "{" stmt+ "}"

expr       ::= INT | FLOAT | STRING | "true" | "false" | ID
             | "-" expr
             | expr ("=="|"!="|">"|"<"|">="|"<="|"++"|"+"|"-"|"*"|"/") expr
             | "new" ID "(" args ")"
             | "now" target "." ID "(" args ")" deadline?
             | "future" target "." ID "(" args ")"
             | "await" expr deadline?
             | "replyto"                  -- the reply destination as a value
             | ID "(" args ")"
             | "(" expr ")"

-- with else it is tau; without else it is result<tau>
deadline   ::= "timeout" INT "else" expr
             | "timeout" INT
```

Operator precedence, weakest first: `== !=` < `> < >= <=` < `++` < `+ -` < `* /` < unary minus. Every binary operator is **left associative** (equality included) — `1 == 2 == false` reads as `((1 == 2) == false)` and evaluates to `true`. Unary minus is a prefix and can be stacked (`- - 3` is `3`).

41 Environment variables of the implementation

Some variables change how checking and execution behave. Set them to `1` to enable; the default is always "not enabled".

Variable	Effect
`AIOS_STRICT_DEADLINE`	make a `now` / `await` without a deadline an **error**; by default it is only a warning. Set it if you want to stay inside the fragment for which the soundness theorem holds (AIPL^{-2})
`AIOS_LAX_OVERLOAD`	restore the **old behaviour** for ambiguous overloads: warn and pick the default candidate. The default is strict (an error)
`AIOS_LAX_EXPOSE`	downgrade a missing annotation on an exposed actor (§21) to a warning. The default is an error
`AIOS_TYPE_TRACE`	print `[rtype] Class#method = tag` on every `reply`; used by the differential test (`scripts/type_runtime_diff.py`) that compares the static return type with the type actually returned
`AIOS_QUIET`	suppress informational messages from the checker and the REPL
`AIOS_STRICT_PROTOCOL`	warn about the "unfinished" part when the steps of a session cannot be laid out statically. Silent by default (§16)
`AIOS_STRICT_RESOURCE`	report `acquire` calls whose name is not a string literal; the order cannot be checked there (§14)
`AIOS_SHOW_LEVELS`	print the inferred obligation levels and the global resource order
`AIOS_LAX_WAIT`	downgrade the detection of circular waits from an error to a warning. The default is an error
`AIOS_LAX_ACTOR`	do not distinguish actor types (unify `actor(A)` with `actor(B)`). The default is to distinguish them
`AIOS_LAX_ASSIGN`	allow assignment to an undeclared name (the old behaviour). The default is an error
`AIOS_MODEL_PROVIDER`	the model vendor used by `ai_call`; the default is `gemini`

For instance, to work under the discipline of always writing deadlines:

```
AIOS_STRICT_DEADLINE=1 ./src/abclrepl_thread -q -f run.repl
```

42 Common pitfalls

- **`now` is an expression.** `now x.m();` is a syntax error; always receive it, as in `var r = now x.m() timeout 100 else 0;`.
- **The target of `send` is a variable**, not a class name.
- **`now self.m()` cannot be written** (it would be a self-deadlock, so the grammar forbids it). To factor out a computation, move it to another actor.
- **You cannot assign an actor to `var x = 0;` later** (it has been inferred as `int`). Create it as `var x = new Foo();` from the start.
- **String concatenation is `++`**; `+` is numeric only.
- **Fields come before methods.**
- **Write a deadline on every wait.** Otherwise you get a warning, and under `AIOS_STRICT_DEADLINE=1` it does not pass at all.
- **Add `?ask=0` when loading onto hardware.** Without it `/actor/loadvm` never answers, which looks like a dead board (§39).
- **`print` on the board goes to the serial line.** To read a value over the network, `web_expose` it and call `/api/x/`.
- **Effects are checked once you declare them** (gradual). Declare nothing and nothing is said; declare something and the difference from the body is pointed at.

43 Present limits

- **What runs on the hardware (Xinu / Pi 3, Pi 4, Pi 5) is a subset of the canonical language.** No `float`, arrays, math builtins, `spawn`, `remote` or `replyto`; type, effect and level annotations are accepted and discarded (checking happens on the canonical side). See §39.
- Splitting a wait into `future` + `await` escapes effect propagation (the "known hole" in §11). Fixing it requires carrying effects in the `future` type.
- There is no variable declaration with a type annotation, `var x : T = e;`.
- `sender` is `any`, so `send sender.m(...)` checks neither the existence of the method nor its types.
- The result of a remote send (`remote(...)`) stays `any`. There is no mechanism for reading the interface description of the peer node.
- Method parameters are monomorphic; polymorphic methods cannot be written.
- There is no implicit conversion between `int` and `float`, and `a + b` with both operands unbound needs an annotation.
- There are no array literals `[...]`; build arrays with `array_empty` and `array_push`.
- A reply destination (`replyto`) **cannot be kept in a field**, though it can be passed as an argument (§13). A field's type comes from its initializer, so `var w = 0;` is `int` and `w = replyto` is a type mismatch.
- The global resource order can only be followed for `acquire` calls whose name is a **string literal** (§14).
- The static session check reaches only as far as **synchronous** callees (§16).
- Termination is not guaranteed. Deadlock freedom means "nothing gets stuck", not "everything finishes". `while 1 > 0 do {}` does not stop, but it is not a wait.

44 Changes from the previous edition

2026-09-06 — the canonical 25 features now run on hardware

The language did not change. What changed is **how much of it the boards accept**.

- `remote("host:port","actor")` (§39.9). The three boards and the Mac host VM can call each other's published actors. **Checked for every pair.** Carried over UDP port 9010; the datagram is shared by all four implementations.
- **First-class replies** (`replyto` / `answer` / parameter type `reply`). On the Pi 3 a reply destination *is* an actor id, so the front end alone was enough; on the Pi 4 and Pi 5 `reply(v)` is the return value, so a reply slot is taken per `now`.
- **Resource order** (`acquire` / `release`). Pairing and order stay canonical; the boards keep a runtime watch. On the Pi 3 it is a **real named semaphore** — actors there are real processes, so two of them can genuinely contend.
- `result` **with** `is_ok` **/** `value`, `float`, arrays and the maths builtins (Pi 4 and Pi 5), and `new K()` in a field initialiser.
- **Only** `spawn` **is left.** The Pi 3 additionally lacks `float`, arrays, the maths builtins and calling one's own method — all of which follow from `.avm` values being 32-bit tagged integers.

2026-09-05 — all three boards accept the same AIPL

The language did not change; the **hardware coverage** did.

- **The Pi 4 was brought up to current AIPL** (§39.11): the on-board front end `abcl2c` was ported, so raw AIPL can be posted to `POST /cc`. **All ten canonical guide samples behave canonically.**
- **`select` now has the same shape on all three** (§39.8). The Pi 5's version did not wait; it was rebuilt on the Pi 3's continuation-split scheme.
- **`wait` works under the Pi 5's `/cc`** (§39.8); there is no longer a path to choose.
- **Booleans became a tagged value** (§39.8): all three print `b = true`. Every board used to print `b = 1`.
- **The Pi 3's output is readable over HTTP** (§39.3). Adding `/api/console` is what finally allowed comparing all ten outputs on the hardware itself.
- Fixed across the boards: `init()` never called for `new C()`, a timed-out call's late reply being taken by the next wait, a double `init`, and actors surviving a program swap.

2026-09-03 — the Pi 5 accepts much more

The language did not change. What changed is **what the Pi 5's on-board front end (`abcl2c`) accepts**; see §39.8.

	Previous	This edition
Canonical samples (unchanged, `POST /cc`)	**0 / 10**	**10 / 10 run**
of which canonical	0	**10**
Annotations `: T` / `!{...}` / `@N`	rejected	accepted, discarded
`reply`	refused	same as `return` (block-final)
`++` / `true` / `false` / unary minus	absent	accepted
Postfix `timeout … else …`	refused	accepted (**checked after**)
`web_listen` / `web_expose`	refused	accepted (`/api/x/`)
`wait(ms)`	refused	accepted (`/cc` too, §39.8)
`future` / `await`	refused	accepted (**not concurrent**)
`select` / `case`	refused	accepted (**waits**)

Several **silent breakages** were fixed along the way: `new` of an unknown class read `CLS[-1]` (an out-of-bounds read on hardware); top-level `var`s referenced from a method produced C that would not compile; field initialisers silently became 0 when they were not numeric; and string literals were truncated at 62 bytes, desynchronising the lexer.

What changed since the previous edition (2026-07-30).

44.1 Removed from the language

	Content
`become`	**removed from the language**. ABCL/1 does not have it; once restricted enough to keep types sound it becomes almost the same as writing a state field and a branch; and not one of 543 real programs used it. Write behaviour replacement with a state field and `if`

44.2 Added

None of these break existing programs. Every annotation may be omitted, and omitting it means it is inferred.

	Content	Section
Failure type	a wait with no `else` returns `result`⟨τ⟩	§12
First-class reply	`replyto` / `answer` / parameter type `reply`, treated linearly	§13
Resource order	the `acquire` / `release` pairs, and the global order read from their nesting	§14
Obligation levels	`@`n, inferred if omitted; also enforced across node borders	§15
Session types	the type checker reads the run-time protocol declarations	§16
The `select` discipline	a mandatory deadline, and a check that a sender exists	§17
Mesh deployment	ship the source and type-check it at the destination before instantiating	§18

44.3 Made stricter

	Content
Assignment to undeclared names	now **rejected**. Previously a new variable was silently created, so a misspelled field name produced no error at all (`AIOS_LAX_ASSIGN=1` restores the old behaviour)
Actor types	`actor(A)` and `actor(B)` are now distinguished (`AIOS_LAX_ACTOR=1` restores the old behaviour)
`reply` totality	a method waited on by `now`/`await` must `reply` on every path, whatever its return type

44.4 The file extension

The source extension is `.aipl` (`.abcl` has been retired completely). The implementation does not look at the extension, so old files still load, but write new ones as `.aipl`.

44.5 Core and extensions — the language has two layers

There are three implementations, but they are **not the same language**.

Layer	Implemented by	Content
Core	all three	the specification this guide describes
Extensions	the Python implementation only	record types, row polymorphism, refinement types via `where`, CSP channels, ownership (`pub`/`move`), tuples, `match`, saga, `scope { future ... }`

This guide specifies the core only. Which layer a program belongs to can be determined mechanically with `tools/classify_corpus.sh` (see `docs/AIPL_LAYERS.md`).

45 Related documents

- `docs/aipl_paper5_ja.pdf` — **the paper (5th edition, latest)**. The current specification, and what was taken from (or left out of) ABCL/1 and Kobayashi's type systems, with the reasons and the trade-offs.

- `coq/AIPLSoundness2.v` — **the machine-checked proof (2nd version)**, in Coq/Rocq. For the core *including* `await`, type soundness, type safety and deadlock freedom are proved with no axioms. The header also discusses the **largest syntax** for which all three hold at once.
- `coq/AIPLSoundness.v` — the 1st version (deadlock freedom only for the fragment without `await`).
- `docs/AIPL_LAYERS.md` — **AIPL has two layers**: the definitions of core (all three implementations) and extensions (Python only), and the classification by `tools/classify_corpus.sh`.
- `docs/aipl_reply_inference_ja.pdf` — return type inference from `reply`, the rationale for annotations, and measurements.
- `docs/aipl_vs_abcl1_ja.pdf` — a comparison with ABCL/1.
- `docs/aipl_abcl1_features_ja.pdf` — the features inherited from ABCL/1.
- `docs/aipl_kobayashi_synthesis_ja.pdf` — a design proposal for taking in the results of Kobayashi's work.
- `docs/ABCLCP_TYPE_SOUNDNESS_REPORT.md` — type soundness (Markdown).
- `docs/AIOS_LANGUAGE_DESIGN.md`, `docs/CAPABILITIES.md` — the design as AIOS and the list of capabilities.
- `docs/samples/reply_inference/README.md` — what the negative samples are for.